SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	SANTHOSH G	**Reg. No.:**	192521035
Date:	20-10-2026	**Duration:**	60 minutes

Code of Conduct

I SANTHOSH G-192521035 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: SANTHOSH G

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. **Understand the Problem Statement**
   - Explain the problem in your own words.
   - Identify the objectives of the proposed system.
   - State the scope of the problem.
2. **Identify Key Stakeholders and Challenges**
   - Identify all the stakeholders involved in the system.
   - Explain the roles and responsibilities of each stakeholder.
   - Discuss the major challenges and issues faced by the stakeholders.
3. **Analyze the Current Approach**
   - Describe the existing system or current process.
   - Identify its strengths and limitations.
   - Analyze the problems or gaps that need to be addressed.
4. **Propose a Suitable Solution Using Software Engineering Principles**
   - Design a software-based solution for the given problem.
   - Describe the major functional modules of the proposed system.
   - Identify the functional and non-functional requirements.
   - Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.
5. **Justify the Chosen Methodology/Framework**
   - Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).

- Justify why the selected methodology/framework is suitable for the proposed system.
- Explain how it supports successful project development and maintenance.

6. **Discuss Expected Outcomes and Limitations**
   - Describe the expected benefits of the proposed solution.
   - Explain how the solution addresses the identified challenges.
   - Discuss potential risks, implementation challenges, and limitations.
   - Suggest possible future enhancements.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/frame work and provides	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing;

	strong technical justification.			report lacks organization and clarity.

Criteria	**Mark**
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25

University Academic Management and Student Success Platform

1. Understanding the Problem Statement

1.1 The Problem in Our Own Words

Universities handle a very large amount of academic information: admissions, course enrolment, daily attendance, internal and external assessments, examination results, faculty workload and student progress. In many institutions this information lives in separate tools such as spreadsheets, standalone admission software, paper registers and email threads. As a result, a student's complete academic picture is not available in one place, reports reach decision-makers late, and a student who is slipping is usually noticed only after failing an examination.

The problem is therefore twofold. First, the university needs one centralised and reliable platform that manages the entire academic lifecycle of a student. Second, the platform must go beyond record keeping: it should continuously watch attendance, marks and enrolment against configurable academic policies, detect academically at-risk students early, and notify faculty and advisors so that timely support can be given.

1.2 Objectives of the Proposed System

- **Centralised academic records:** maintain a single, accurate and secure record for every student, course and faculty member.
- **Monitoring:** track attendance and academic performance continuously and in near real time.
- **Course registration:** allow online, rule-checked enrolment with seat and prerequisite validation.
- **Examination management:** support scheduling, hall allocation, mark entry, moderation, result publishing and revaluation.

- **Early identification:** flag academically at-risk students using configurable institutional policies.
- **Communication:** notify students, faculty, advisors and administrators automatically.
- **Reporting and planning:** generate academic reports and dashboards that support data-driven decisions.
- **Engagement and efficiency:** improve student engagement, reduce manual work and increase transparency.

1.3 Scope of the Problem

In scope: student admissions, course registration, attendance, assessments and examinations, academic records and transcripts, faculty academic activities, at-risk detection and advising workflow, notifications, institutional reports and dashboards, role-based access, and support for multiple departments and programmes with thousands of concurrent users.

Out of scope (for this report): detailed finance and fee accounting, hostel and transport management, payroll and HR, library management and placement management. These may be integrated later through APIs. The system is expected to be web-based, mobile-friendly and deployable on cloud or on-premise infrastructure.

2. Key Stakeholders and Challenges

2.1 Stakeholders, Roles and Responsibilities

Stakeholder	Role and responsibilities	Key needs from the system
Students	Apply for admission, register for courses, view attendance, marks, timetable and results, respond to alerts and raise grievances.	Transparent, real-time view of progress; easy registration; timely alerts.
Faculty	Mark attendance, set assessments, enter and moderate marks, monitor class performance, mentor students.	Quick data entry, class-level insights, early list of struggling students.
Academic advisors / mentors	Review at-risk students, plan and record interventions, counsel students.	Consolidated student profile, alerts, intervention tracking.
Department heads / HoDs	Plan course offerings, allocate faculty, approve results, review departmental performance.	Departmental dashboards and trend reports.
Administrators and registrar	Manage admissions, academic calendar, records, transcripts, policies and user accounts.	Accurate records, automation of routine work, audit trail.
Examination controller / cell	Schedule exams, manage invigilation, ensure confidentiality, publish results.	Secure, error-free exam workflow.
Senior management (Dean, VC)	Strategic planning, accreditation and quality assurance.	Institution-wide analytics and compliance reports.

Stakeholder	Role and responsibilities	Key needs from the system
Parents / guardians (optional)	Follow the ward's attendance and performance.	Limited, consent-based view and notifications.
IT and support staff	Deploy, secure, maintain and support the platform.	Maintainable, well-documented, monitorable system.
External bodies (regulators, accreditation agencies)	Require standardised academic data and reports.	Exportable, audit-ready reports.

2.2 Major Challenges Faced by the Stakeholders

- **Students:** scattered information, delayed results, registration clashes, lack of early feedback on poor attendance or marks.
- **Faculty:** repetitive manual attendance and mark entry, difficulty tracking individual student progress in large classes, heavy report-preparation load.
- **Advisors:** no early warning; problems are discovered late, and there is no structured record of interventions.
- **Administrators:** duplicated data across systems, errors in manual entry, time-consuming report compilation, difficulty enforcing regulations uniformly.
- **Management:** limited visibility into trends such as dropout rates, pass percentages and course demand, which weakens planning.
- **Institution as a whole:** scalability during peak periods (admissions, registration, results), data security and privacy, system downtime and compliance requirements.

3. Analysis of the Current Approach

3.1 Existing System / Current Process

In most universities the present process is partly manual and partly digital. Admissions are processed through forms and separate software. Course registration is done on paper or through a basic portal. Attendance is recorded in paper registers or spreadsheets and consolidated by the department at the end of the month or semester. Marks are collected in spreadsheets, compiled by the examination cell and published as PDF lists. Reports for management are prepared manually by collecting files from each department, and communication with students happens through notice boards, emails and messaging groups.

3.2 Strengths of the Current Approach

- Familiar to staff, so there is little training required.
- Low initial cost and no dependence on a single software vendor.
- Flexible: departments can adapt their own spreadsheets and formats quickly.

- Existing departmental know-how and personal contact with students supports informal mentoring.

3.3 Limitations of the Current Approach

- Data silos and duplication: the same student data is stored in many places and often becomes inconsistent.
- High manual effort and a high chance of human error in attendance, marks and results.
- Delayed reporting: information reaches decision makers after the opportunity to act has passed.
- No proactive monitoring: at-risk students are identified only after failure.
- Weak security and audit trail: spreadsheets and paper files are easy to alter or lose.
- Poor scalability: processes break down as student numbers and programmes grow.
- Inconsistent policy enforcement because rules are applied manually and differently by each department.

3.4 Gap Analysis

Area	**Current state**	**Gap to be addressed**
Student records	Fragmented across departments	Single source of truth with unique student ID
Attendance	Paper / spreadsheet, monthly consolidation	Digital, real-time capture with automated shortage alerts
Assessment and exams	Manual compilation of marks	Workflow with validation, moderation and secure publishing
At-risk identification	Reactive, based on individual teachers' judgement	Rule-based automated early-warning engine
Notifications	Notice boards and ad hoc messages	Automated multi-channel (email, SMS, app) alerts
Reporting	Manual, slow and error prone	Self-service dashboards and scheduled reports
Security and availability	Limited access control, no audit trail	Role-based access, encryption, audit logs, high availability
Scalability	Fails at peak load	Elastic architecture supporting thousands of concurrent users

4. Proposed Solution Using Software Engineering Principles

4.1 Overview of the Solution

We propose a web-based, role-driven University Academic Management and Student Success Platform built on a modular, layered architecture. A presentation layer (web portal and mobile-responsive interface) talks to an application layer made of independent modules exposed through secure REST APIs. A central relational database stores academic records, while a rules engine applies configurable academic policies, and an analytics layer powers dashboards and the at-risk prediction. A notification service delivers email,

SMS and in-app alerts. The platform is deployed on cloud infrastructure behind a load balancer with database replication and automated backups.

4.2 Major Functional Modules

Module	Description
User and Access Management	Authentication, single sign-on, role-based access control (student, faculty, advisor, HoD, admin, exam cell) and audit logging.
Admission Management	Online application, document upload, merit list generation, verification, seat allotment and generation of student ID.
Course and Programme Management	Maintain departments, programmes, curricula, credits, prerequisites, timetable and academic calendar.
Course Registration	Online enrolment with seat limits, prerequisite and clash checks, add/drop windows and advisor approval.
Attendance Management	Faculty marks attendance through web/mobile; supports QR or biometric integration; computes percentages and shortage.
Assessment and Examination	Internal assessments, assignment marks, exam scheduling, hall tickets, mark entry, moderation, grade and GPA calculation, result publishing, revaluation.
Academic Records	Permanent student record, transcripts, certificates, degree audit and history of changes.
Faculty Activity Management	Course allocation, workload, teaching plans, mentoring records, research and academic activities.
Student Success and Early-Warning Engine	Applies configurable policies (e.g., attendance below threshold, falling marks, failed courses, low engagement) to flag at-risk students, assign risk level and trigger interventions.
Notification and Communication	Event-driven alerts and announcements to students, faculty and advisors through multiple channels.
Reports and Analytics	Dashboards and downloadable reports on pass percentage, attendance trends, course demand, faculty performance and retention.
Policy Configuration	Admin-controlled rules for grading, attendance, eligibility and risk thresholds without changing code.

4.3 Functional Requirements

- FR1: The system shall allow students to apply for admission online and track application status.
- FR2: The system shall allow admins to create and maintain programmes, courses, prerequisites and sections.
- FR3: The system shall allow students to register, add or drop courses within the registration window, and shall reject clashes, seat overflow and missing prerequisites.
- FR4: The system shall record daily attendance and calculate attendance percentage per course and per student.
- FR5: The system shall allow faculty to define assessment components and enter marks, and shall compute grades and GPA/CGPA automatically.

- FR6: The system shall manage the examination workflow from scheduling to result publication, including revaluation requests.
- FR7: The system shall maintain a complete academic record and generate transcripts and certificates.
- FR8: The system shall evaluate configurable policies and identify at-risk students automatically at defined intervals.
- FR9: The system shall notify students, faculty and advisors when risk or policy thresholds are breached or when results and notices are published.
- FR10: The system shall let advisors record interventions and follow up on at-risk students.
- FR11: The system shall provide role-specific dashboards and exportable reports (PDF, Excel).
- FR12: The system shall enforce role-based access and maintain an audit trail of all changes to academic data.

4.4 Non-Functional Requirements

Attribute	**Requirement**
Performance	Common pages respond within 2 seconds under normal load; registration and result peaks handled without degradation.
Scalability	Support at least 50,000 student records and several thousand concurrent users; horizontal scaling of application servers.
Availability and reliability	99.9% uptime, redundant servers, database replication, automated daily backups and disaster recovery plan.
Security	Encrypted data in transit (TLS) and at rest, strong authentication with optional multi-factor, protection against OWASP Top 10 threats, privacy compliance.
Usability	Simple, consistent and accessible interface (WCAG guidelines), mobile-responsive, multilingual support where required.
Maintainability	Modular code, coding standards, documentation, automated tests, and configuration-driven policies.
Interoperability	Open REST APIs for integration with LMS, library, finance and government/accreditation systems.
Auditability and compliance	Immutable logs of grade changes and record edits; data retention as per university regulations.

4.5 Software Engineering Principles Incorporated

Principle	**How it is incorporated in the solution**
Modularity	The system is divided into loosely coupled modules (admission, registration, attendance, exams, alerts, reports) with well-defined APIs, allowing independent development, testing and replacement. High cohesion within modules and low coupling between them.
Scalability	Stateless application servers behind a load balancer, database indexing and read replicas, caching of frequent queries, asynchronous queues for notifications and report generation, and cloud auto-scaling for peak periods.

Principle	How it is incorporated in the solution
Maintainability	Layered architecture, separation of concerns, clean code and design patterns, version control, code reviews, automated unit and regression tests, and policy rules stored as configuration so that regulation changes need no redeployment.
Reliability	Input validation, transaction management for registration and marks, retry mechanisms for notifications, redundant deployment, automated backups, monitoring and failover.
Usability	Role-specific dashboards, minimal-click workflows, clear error messages, responsive design and user testing with students and faculty during each sprint.
Security	Role-based access control, least privilege, encryption, secure coding, audit trails, periodic vulnerability testing and data privacy safeguards.
Reusability and extensibility	Shared services (authentication, notification, reporting) are reused across modules; new departments or programmes are added through configuration.

5. Justification of the Chosen Methodology

5.1 Selected Model: Agile (Scrum) with DevOps Practices

For this project we select the Agile Scrum framework supported by DevOps practices such as continuous integration and continuous delivery (CI/CD). The product is developed in short sprints of two to three weeks, each producing a working increment, for example Sprint 1 for user management and records, Sprint 2 for course registration, Sprint 3 for attendance, and so on.

5.2 Why It Is Suitable

- **Evolving requirements:** academic regulations, grading schemes and risk rules differ between departments and change often. Agile welcomes change, whereas Waterfall would fix requirements too early.
- **Multiple stakeholders:** students, faculty, advisors and administrators can review each increment in sprint reviews and give feedback, improving usability and adoption.
- **Early value and risk reduction:** core modules such as records and registration can be released first, and risky parts such as the at-risk engine can be validated with real data in early iterations.
- **Large and complex system:** the modular architecture fits naturally with a prioritised product backlog and incremental delivery.
- **Quality and security:** continuous integration, automated testing and regular code reviews detect defects early.
- **Seasonal deadlines:** releases can be planned around the academic calendar (admissions, registration, exams).

5.2.1 Comparison with Other Models

Model	Advantage	Why it is less suitable here
Waterfall	Clear documentation and phases	Inflexible to changing policies; stakeholders see the product only at the end.
Spiral	Strong risk analysis	Costly and complex to manage for a routine administrative domain.
Agile Scrum + DevOps	Iterative delivery, feedback, quick adaptation, automation	Selected: best balance of flexibility, quality and speed.

5.3 Support for Project Development and Maintenance

- Product owner (registrar or academic dean) maintains the prioritised backlog, ensuring that the most valuable features are built first.
- Sprint planning, daily stand-ups, sprint reviews and retrospectives keep the team aligned and continuously improving.
- CI/CD pipelines with automated build, test and deployment shorten release cycles and reduce deployment errors.
- Definition of Done includes testing, security checks and documentation, which supports long-term maintainability.
- After go-live, the same backlog-driven cycle handles bug fixes, new regulations and enhancements, so maintenance continues smoothly without a separate process.

6. Expected Outcomes and Limitations

6.1 Expected Benefits

- Improved academic administration through a single, accurate and secure record system.
- Higher student success and retention because at-risk students are identified and supported early.
- Increased faculty productivity and reduced manual administrative work through automation.
- Greater transparency: students and staff can see attendance, marks and status at any time.
- Timely academic interventions supported by automated alerts and advisor workflows.
- Better institutional efficiency, communication and evidence-based academic planning through dashboards.
- Scalable university management that can accommodate new departments, programmes and growing enrolment.

6.2 How the Solution Addresses the Identified Challenges

Challenge	How it is addressed
Fragmented data and duplication	Centralised database with unique student ID and integrated modules.

Challenge	How it is addressed
Delayed reporting	Real-time dashboards and scheduled automated reports.
Late identification of struggling students	Policy-driven early-warning engine with alerts to students, faculty and advisors.
Manual errors and workload	Automated calculations, validations and online data entry.
Poor communication	Event-driven multi-channel notification service.
Security and audit concerns	Role-based access, encryption and audit logs.
Scalability and downtime	Cloud-based elastic and redundant architecture.

6.3 Risks, Implementation Challenges and Limitations

Risk / limitation	Description	Mitigation
Resistance to change	Staff and students used to existing methods may resist adoption.	Training, phased rollout, champions in each department.
Data migration	Legacy records may be incomplete or inconsistent.	Data cleansing, validation scripts, pilot migration.
Security and privacy	Sensitive student data is a target for breaches.	Encryption, access control, regular security audits.
Peak-load failures	Registration and results days create traffic spikes.	Load testing, auto-scaling, caching.
Prediction accuracy	Rule-based risk detection may produce false positives or miss non-academic causes.	Tune thresholds, advisor review, combine with human judgement.
Cost and effort	Initial development, infrastructure and training need investment.	Phased delivery, prioritised backlog, cloud pay-as-you-go.
Integration complexity	Linking with existing finance, library or LMS systems may be difficult.	Standard APIs and adapters.
Dependence on connectivity	Online system requires reliable internet and power.	Offline-tolerant mobile features, backup infrastructure.

6.4 Possible Future Enhancements

- Machine-learning based predictive analytics for dropout and performance forecasting.
- AI chatbot for student queries and an intelligent course recommendation engine.
- Mobile application with offline attendance capture and push notifications.
- Integration with learning management systems, library, finance and placement modules.
- Biometric or face-recognition attendance and online proctored examinations.
- Blockchain-based tamper-proof digital certificates and credentials.
- Learning analytics and personalised learning paths, and multilingual and voice accessibility support.

6.5 Conclusion

The proposed University Academic Management and Student Success Platform replaces fragmented, manual academic processes with an integrated, secure and scalable system. By applying modular design and sound software engineering principles, and by developing the system through Agile Scrum with DevOps practices, the university can deliver value early, adapt to changing regulations, and move from reactive record keeping to proactive student support, ultimately improving student success and institutional efficiency.